Conversación con Claude — TPI MS2: preparación del trabajo

Integrante: Cristian Morales
Herramienta: Claude (Anthropic), sesión de trabajo en claude.ai
Período: 17/10/2026 18:11 a 17/10/2026 21:38 (hora de Argentina) — 38 mensajes
Fuente: exportación oficial de datos de la cuenta (Settings → Privacy → Export data), conversación «Sistema de evaluación y setup»
Nota: se transcribe el texto visible de cada mensaje. Se omiten el razonamiento interno del modelo y el detalle técnico de las herramientas; en su lugar se indica la acción realizada (archivos creados, comandos ejecutados, archivos enviados). Las capturas de pantalla adjuntas están en la carpeta de evidencias del repositorio. Los mensajes posteriores a la exportación no están incluidos.
Parte: 1 de 2 — Preparación del TP (consigna, sistema base, repositorio, organización del equipo y plan de trabajo). Continúa en «cristian_claude_02_diagnostico.pdf».

Usuario (Cristian Morales) 17/10/2026 18:11

tengo que empezar a encarar este tp con compañeros,

dame un resumen de que hay que hacer y como armarlo, la idea es resolverlo con tu ayuda pero no que lo armes

[Archivo adjunto: TP MS2V2.pdf]

Claude 17/10/2026 18:12

Acá va el panorama para que lo encaren en equipo.

La idea central

No se trata de construir un sistema, sino de **tomar uno chico que funcione pero esté mal diseñado, diagnosticarlo, mejorarlo y demostrar que sigue funcionando igual**. La nota sale de la calidad del análisis y la trazabilidad: qué problema encontraron, por qué importa, qué hicieron y cómo prueban que no rompieron nada.

Qué tiene que tener el sistema base

- Al menos **3 entidades** (por ejemplo Solicitud, Estudiante, Responsable/Área, Estado).
- Al menos **4 operaciones verificables** (registrar, cambiar estado, asignar, filtrar, reportar).
- **Persistencia**, que puede ser archivo, base de datos o memoria.
- **Problemas de diseño reales** que se puedan diagnosticar.

Pueden usar el caso sugerido (Solicitudes Académicas) o un proyecto propio. Una opción natural es algo que ya hayan hecho en Programación IV, como el Food Store con pedidos. Eso sí: si ese código ya está prolijo, les va a costar encontrar 8 smells genuinos. Conviene un sistema que tenga "mugre" real: funciones largas, lógica en el lugar equivocado, duplicación.

Las 8 etapas, en orden de trabajo

1. **Sistema base**: describir dominio, actores, funcionalidades y stack. Lo congelan en git con el tag `estado-inicial`.
2. **Calidad externa vs. interna**: qué ve el usuario (funciona) frente a qué hay adentro (difícil de mantener). Necesitan ejemplos concretos.
3. **Deuda técnica y code smells (20%)**: mínimo 8. Les conviene una tabla con columnas *smell / evidencia / ubicación / impacto / riesgo / prioridad*. Es una de las etapas que más pesa.

4. **SOLID y Clean Code (20%)**: vincular cada mejora a un principio (SRP, OCP, DIP, etc.) y a nombres, funciones, manejo de errores y acoplamiento. Solo apliquen los que correspondan, no fuercen los cinco.
5. **Refactoring seguro (20%)**: mínimo 4, y obligatoriamente estos tres:
   - Extract Method
   - Move Method (o reubicar una responsabilidad)
   - Introduce Parameter Object (o una simplificación de parámetros equivalente)

   Cada refactoring va con su código antes/después y su commit.
6. **Pruebas (10%)**: los casos de prueba se escriben **antes** de refactorizar y se corren antes y después. Tests automatizados (pytest, jest, etc.) son la mejor evidencia.
7. **Code review (10%)**: armar un checklist propio, aplicarlo sobre el repo y registrar observaciones, correcciones y criterios de aceptación. Hacerlo con Pull Requests en GitHub, revisándose entre ustedes, deja evidencia sola.
8. **Conclusiones**: grupales y también individuales breves.

Entregables (los cuatro son obligatorios)

- **PDF único**: portada, resumen ejecutivo de 5 a 7 líneas, las etapas con capturas y tablas, registro de prompts de IA, links al repo y al video.
- **Repo GitHub** con:
  - Tags generales `estado-inicial` y `estadorefactorizado`. Este último parece tener un typo en la consigna; consúltenlo para no fallar en algo tonto.
  - Tags de hito: `etapa-diagnostico`, `etapa-refactoring`, `etapa-pruebas`, `etapa-docs`.
  - README, una carpeta `/docs` con el checklist y las decisiones técnicas, y commits claros de **todos** los integrantes.
- **ZIP de anexos**: capturas, diagramas, informes de herramientas y conversaciones con IA.
- **Video de 10 a 15 minutos**: todos con cámara y nombre al inicio, voz propia.

Cómo organizarse

Un orden que funciona bien:

1. **Juntada inicial**: elegir el sistema base, crear el repo y definir convenciones (ramas, mensajes de commit, PRs obligatorios).
2. **Diagnóstico en paralelo**: cada uno revisa una parte del código y propone smells. Después unifican la tabla y priorizan.
3. **Pruebas primero**: escribir los tests sobre la versión inicial antes de tocar nada.
4. **Refactor repartido**: cada integrante toma uno o dos refactorings, en su propia rama, con un PR que revisa otro compañero. Así cubren la etapa 7 y la "participación demostrable" al mismo tiempo.
5. **Documentar sobre la marcha**: tomar capturas y guardar prompts mientras trabajan, no al final.
6. **Cierre**: armar el PDF, grabar el video y revisar que todos los links abran.

Ojo con esto

- **IA**: está permitida pero hay que declararla (herramienta, finalidad, grado de intervención) y adjuntar la evidencia. Esta conversación misma entra en el anexo, así que vayan guardando los chats relevantes.
- **Datos**: solo sintéticos, nada de datos reales de personas.
- **Coherencia**: cada smell debería conectar con una mejora, cada mejora con un principio y cada cambio con una prueba. Esa cadena es lo que más evalúan.
- **Defensa individual**: cualquiera puede ser preguntado sobre cualquier parte, así que todos tienen que entender todo.

El primer paso concreto es **elegir el sistema base**. Cuando lo tengan (o si quieren que comparemos candidatos), lo vemos y los ayudo a detectar los smells.

Usuario (Cristian Morales) 17/10/2026 19:52

podrias armarnos un sistema base con las condiciones necesarias para hacer la evaluacion?

Claude 17/10/2026 19:55

Antes de armarlo necesito definir dos cosas que cambian todo el código, así no lo rehacemos después.

Pregunta al usuario

- **¿En qué stack lo armo? Tiene que ser algo que todo el grupo pueda leer, correr y refactorizar.**
  Opciones: Python + FastAPI + SQLite (Recommended), Python consola + JSON, Node + TypeScript + Express
- **¿Qué dominio usamos?**
  Opciones: Solicitudes Académicas (Recommended), Food Store con pedidos, Turnos / reservas

Respuesta: Your questions have been answered: "¿En qué stack lo armo? Tiene que ser algo que todo el grupo pueda leer, correr y refactorizar."="Python + FastAPI + SQLite (Recommended)", "¿Qué dominio usamos?"="Solicitudes Académicas (Recommended)". You can now continue with these answers in mind.

Armo el sistema base: FastAPI + SQLite, dominio Solicitudes Académicas, con problemas de diseño reales metidos a propósito pero sin señalarlos, para que el diagnóstico lo hagan ustedes.

Creó el archivo `main.py`

Creó el archivo `README.md`

Creó el archivo `requirements.txt`

Creó el archivo `.gitignore`

Ahora verifico que todo funcione de punta a punta.

Ejecutó un comando: Instalar dependencias y probar todos los endpoints

Ejecutó un comando: Instalar dependencias y probar todos los endpoints

Todo funciona. Lo empaqueto.

Ejecutó un comando: Comprimir el proyecto en un zip

Envió el archivo `solicitudes-academicas.zip`

Listo: el sistema base está en **solicitudes-academicas.zip**. Lo probé de punta a punta y todas las operaciones funcionan.

Qué cubre de los requisitos mínimos

- **4 entidades**: Estudiante, Responsable/Área, Solicitud e Historial de estados.
- **10 endpoints**: altas y listados, registrar solicitud, cambiar estado, asignar responsable, buscar/filtrar, detalle con historial y reporte.
- **Persistencia en SQLite**, con datos sintéticos que se cargan solos la primera vez.
- **Problemas de diseño reales**, puestos a propósito pero sin señalar en el código. Encontrarlos es la etapa 3, así que no les paso la lista. Hay de sobra para llegar a 8 y alcanzan para los tres refactorings obligatorios. Cuando tengan su diagnóstico, si quieren lo contrastamos.

Para arrancar

1. Descomprimir, crear el repo y hacer el primer commit con el tag `estado-inicial`.
2. Correrlo (`uvicorn main:app --reload`) y abrir `/docs`. Ahí ya tienen evidencia de funcionamiento para la etapa 1.
3. Antes de tocar nada, escribir los tests sobre esta versión. Son la red que después demuestra que el refactor no rompió nada.

Un punto importante: este código lo generé yo, así que en el informe tienen que declarar que el sistema base fue construido con asistencia de IA y adjuntar esta conversación en el anexo. La consigna

lo permite si se declara. Además, como pueden pedir defensa individual, conviene que cada uno lo lea completo y lo entienda antes de empezar el diagnóstico.

Usuario (Cristian Morales) 17/10/2026 20:11

source : El término 'source' no se reconoce como nombre de un cmdlet, función, archivo de script o programa ejecutable. Compruebe si escribió correctamente el nombre o, si incluyó una ruta de acceso, compruebe que dicha ruta es correcta e inténtelo de nuevo.
En línea: 1 Carácter: 1

- source .venv/bin/activate # Windows: .venv\Scripts\activate
- ```
  + CategoryInfo          : ObjectNotFound: (source:String) [], CommandNotFoundException
  + FullyQualifiedErrorId : CommandNotFoundException
  ```

Claude 17/10/2026 20:11

Estás en PowerShell (Windows) y `source` es de Linux/Mac. En PowerShell el comando para activar el entorno es:

```
.venv\Scripts\Activate.ps1
```

Si te devuelve un error de "la ejecución de scripts está deshabilitada en este sistema", habilitalo solo para esa terminal y volvé a activar:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.venv\Scripts\Activate.ps1
```

Cuando esté activo vas a ver `(.venv)` al principio de la línea. Después seguís igual:

```
pip install -r requirements.txt
uvicorn main:app --reload
```

Si alguno del grupo usa CMD en vez de PowerShell, el comando es `.venv\Scripts\activate.bat`.

Conviene corregir el README para que tenga los comandos de cada sistema por separado. Así le sirve a todo el grupo y además suma al code review.

Usuario (Cristian Morales) 17/10/2026 20:11

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload
```

ajustame esto para power shell

Claude 17/10/2026 20:11

```
python -m venv .venv
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
uvicorn main:app --reload
```
```

La segunda línea habilita los scripts solo en esa terminal, así no te frena el `Activate.ps1`. Si ya los tenés habilitados, podés saltearla.

Usuario (Cristian Morales) 17/10/2026 20:14

no entendi si hay que armar un repositorio

Claude 17/10/2026 20:14

Sí, el repositorio en GitHub es uno de los cuatro entregables obligatorios (PDF, repo, ZIP y video). Además, la cátedra mira el historial de commits y los tags para ver qué hizo cada uno, así que conviene armarlo ya, antes de tocar el código.

Cómo armarlo

1. Uno del grupo crea el repo en GitHub

- En github.com: **New repository**, por ejemplo `tpi-solicitudes-academicas`.
- Público, o privado si después le dan acceso al docente.
- **Sin** README ni .gitignore, porque ya vienen en el zip.

2. Sube el sistema base tal cual, sin cambios

Desde la carpeta del proyecto, en PowerShell:

```
git init
git add .
git commit -m "Sistema base inicial"
git branch -M main
git remote add origin https://github.com/USUARIO/tpi-solicitudes-academicas.git
git push -u origin main
git tag estado-inicial
git push origin estado-inicial
```

Reemplazá `USUARIO` por el usuario de GitHub de quien lo creó.

3. Invita al resto del grupo

- En el repo: **Settings → Collaborators → Add people**, con el usuario de GitHub de cada compañero.

4. Cada compañero lo clona

```
git clone https://github.com/USUARIO/tpi-solicitudes-academicas.git
```

Cómo trabajar después

- Cada cambio va en su propia rama (`git checkout -b refactor-extract-method`, por ejemplo).
- Se sube con un Pull Request y **otro integrante lo revisa** antes de mergear. Eso les deja la evidencia del code review hecha.
- Al cerrar cada etapa, ponen el tag que pide la consigna: `etapa-diagnostico`, `etapa-refactoring`, `etapa-pruebas`, `etapa-docs` y, al final, `estadorefactorizado`.

Lo importante es que el primer commit sea el código **sin tocar**, porque esa es la versión "antes" contra la que van a comparar todo.

Usuario (Cristian Morales) 17/10/2026 20:31

estas etapas que mencionas estaria bueno que la encare una cada uno?

Claude 17/10/2026 20:31

No es lo ideal, por dos motivos:

- **Las etapas dependen una de otra.** Sin diagnóstico no hay qué refactorizar, sin tests no se puede refactorizar seguro y sin refactor no hay qué revisar. Si cada uno tiene una etapa, el de refactoring espera al de diagnóstico, el de review espera a todos, y la carga queda despareja.
- **La evaluación pide participación de todos en todo.** Puede haber defensa individual sobre *cualquier* parte, y en el video y en el repo se tiene que ver qué hizo cada uno. Si a vos te toca "pruebas" y te preguntan por un refactoring que no tocaste, quedás expuesto.

Lo que funciona mejor: repartir el código, no las etapas

Todos pasan por todas las etapas, pero cada uno sobre **una parte distinta del sistema**. Con 4 integrantes, por ejemplo:

Integrante	Parte del código
A	Registrar solicitud
B	Cambio de estado y asignación
C	Búsqueda y reporte
D	Estudiantes, responsables y la conexión a la base

Cada uno, sobre su parte:

1. Detecta smells y aporta a la tabla común.
2. Escribe los tests de esa funcionalidad sobre la versión inicial.
3. Hace sus refactorings en su rama.
4. Revisa el PR de **otro** compañero.

Además, un responsable por entregable

Aparte de su parte del código, cada uno coordina un entregable para que nada quede huérfano:

- **A:** tabla de smells y priorización (que quede unificada y sin repetidos).
- **B:** checklist de code review y carpeta `/docs`.
- **C:** suite de tests y evidencias antes/después.
- **D:** armado del PDF, tags del repo y edición del video.

Coordinar no significa hacerlo solo: es juntar lo de todos y cuidar que esté completo.

Si son menos de 4, se agrupan las partes del código: con 3, por ejemplo, C se queda con búsqueda, reporte y lo de D.

Un detalle: algunos refactorings cruzan todo el código. Sacar la conexión global o crear una función común para convertir filas, por ejemplo, afecta a todos. Esos conviene hacerlos **primero** y mergearlos antes de que el resto arranque, así no se pisan con conflictos.

Usuario (Cristian Morales) 17/10/2026 20:37

entonces hagamos esto, separa que parte deberia ver cada uno y los pasos a seguir para cada integrante (si son pasos generales, habria que armar un checklist de lo que tiene que hacer cada uno)

Claude 17/10/2026 20:38

Lo armo como un archivo Markdown para subir al repo en `docs/`. Así lo tiene todo el grupo, GitHub muestra las casillas de cada checklist y de paso les queda como evidencia de cómo se organizaron.

Creó el archivo `PLAN_DE_TRABAJO.md`

Envió el archivo `PLAN_DE_TRABAJO.md`

En **PLAN_DE_TRABAJO.md** está todo armado. Va en la carpeta `docs/` del repo.

Tiene seis partes:

- **Reparto:** qué funciones y líneas de `main.py` le tocan a cada uno, qué coordina cada uno y quién revisa a quién.
- **Orden de trabajo:** seis fases, con el tag que cierra cada una.
- **Checklist grupal:** lo que hacen juntos, como la reunión de diagnóstico y el cierre.
- **Checklist individual:** el mismo para los 4, aplicado sobre su parte, desde el diagnóstico hasta el video.
- **Checklist de coordinación:** uno distinto para cada rol.
- **Registro de uso de IA:** una tabla que cada uno completa con lo suyo.

El checklist individual dice *qué* hacer (por ejemplo, detectar al menos 3 smells con sus 6 campos), pero no *cuáles* son los smells, porque eso es el diagnóstico que les toca a ustedes. Además, les asigné a cada uno uno de los refactorings obligatorios para que no quede ninguno sin cubrir.

Antes de arrancar, reemplacen A, B, C y D por los nombres. Si son 3, abajo de la tabla de reparto está cómo se redistribuye.

Usuario (Cristian Morales) 17/10/2026 20:45

qie cambios podriamos afrontar ahora para evitar que haya problemas a futuro

Claude 17/10/2026 20:45

Lo primero es una aclaración: **el código de `main.py` no se toca hasta tener el tag `estado-inicial`**. Si lo arreglan ahora, pierden la versión "antes" que necesitan para todo el trabajo.

Lo que sí conviene resolver ya es lo que rodea al código: la configuración del repo y los acuerdos del equipo. Eso no afecta al diagnóstico y les ahorra problemas después.

1. Ajustes del repo (en los primeros commits)

- **README con instrucciones para Windows.** Ya vieron que lo de `source` no anda en PowerShell. Pongan los comandos para Windows y para Linux/Mac por separado.
- **Archivo `.gitattributes`** con `* text=auto`. Si algunos usan Windows y otros Linux/Mac, los finales de línea distintos hacen que Git marque archivos enteros como modificados aunque nadie haya tocado nada. Eso ensucia los diffs del antes/después.
- **`requirements-dev.txt`** con `pytest` y `httpx`, para que todos usen las mismas herramientas de test.
- **Plantilla de Pull Request** (`.github/pull_request_template.md`) con campos como: smell que resuelve, técnica aplicada, principio SOLID y tests corridos. Así cada PR ya queda documentado para el informe.
- **Proteger la rama `main`** (Settings → Branches): que no se pueda pushear directo y que todo entre con un PR aprobado por otro. Esto obliga a hacer code review sin depender de la voluntad de nadie.

2. Acuerdos del equipo

- **Misma versión de Python** para todos (3.11 o 3.12, por ejemplo).
- **Desactivar el formateo automático al guardar** (Black, autopep8, Prettier) hasta la fase de refactoring. Si a uno el editor le reformatea `main.py` al abrirlo, el diff muestra cientos de líneas cambiadas que no son refactoring y se pierde la trazabilidad. Si quieren formatear, que sea un commit aparte y declarado.
- **Convención de commits**, por ejemplo `docs:`, `test:`, `refactor:`, `chore:`. Así el historial se lee solo.
- **Nombres de ramas**, por ejemplo `test/registrar`, `refactor/estado`.

3. Lo que más problemas puede traer: los conflictos

Los 4 van a trabajar sobre **un único archivo**, así que si refactorizan en paralelo sin coordinarse, van a tener conflictos en cada merge. Para evitarlo:

- **Antes de la Fase 3**, acuerden entre todos la estructura de carpetas y módulos a la que quieren llegar. Esa es una decisión de diseño que sale de su diagnóstico, y conviene que quede escrita en `/docs` como decisión técnica.
- Los cambios transversales los hace uno solo, se mergean, y **recién después** arranca el resto.
- Cada uno actualiza su rama desde `main` antes de abrir el PR.

4. Desde el día uno

- **Registro de IA:** que cada uno vaya guardando los chats que use. Reconstruirlo al final es muy difícil.
- **Carpeta de evidencias** con nombres claros, como `fase1_registrar_ok.png` o `fase2_tests_antes.png`.
- **Tests con base limpia:** como el sistema guarda los datos en un archivo, si corren los tests dos veces sobre la misma base los resultados cambian. Definan desde el principio cómo arranca cada corrida con los datos iniciales.

Si quieren, les preparo el `.gitattributes`, el `requirements-dev.txt` y la plantilla de PR para que los suban en el primer commit, junto con el README corregido.

Usuario (Cristian Morales) 17/10/2026 20:47

Skip to content

1. CMorales2703
2. TPI_MS2_Mazza_Morales_Ovelar_Paschetta

Repository navigation

-
- Code
- Issues
- Pull requests
- Agents
- Actions
- Projects
- Wiki
- Security and quality
- Insights
- Settings

Settings: CMorales2703/TPI_MS2_Mazza_Morales_Ovelar_Paschetta

-
-
- Access
  *
  *
  *
  *
-
- Code, planning, and automation
  *
  *
  *
  *
  *
  *
  *
  *
  *

    *
    *
    *
    *
    *
    *
    *
    *
    *
    *
    *
    *
-
- Security and quality
    *
    *
    *
    *
    *
    *
    *
-
- Integrations
    *

Rulesets/New branch ruleset
Protect your most important branches
Rulesets define whether collaborators can delete or force push and set requirements for any pushes, such as passing status checks or a linear commit history.
Ruleset Name*
Enforcement status
Bypass list
Exempt roles, teams, agents, apps, and users from this ruleset by adding them to the bypass list.
Actors

- All
- Roles
- Teams
- Apps
- Users
- Others

Bypass list is empty
Target branches
Which branches should be matched?
Branch targeting criteria
Branch targeting has not been configured
Rules
Which rules should be applied?
Branch rules
Restrict creationsOnly allow users with bypass permission to create matching refs.
Restrict updatesOnly allow users with bypass permission to update matching refs.
Restrict deletionsOnly allow users with bypass permissions to delete matching refs.
Require linear historyPrevent merge commits from being pushed to matching refs.
Require deployments to succeedChoose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.

Require signed commitsCommits pushed to matching refs must have verified signatures.
Require a pull request before mergingRequire all commits be made to a non-target branch and submitted via a pull request before they can be merged.
Require status checks to passChoose which status checks must pass before the ref is updated. When enabled, commits must first be pushed to another ref where the checks pass.

- Require branches to be up to date before mergingWhether pull requests targeting a matching branch must be tested with the latest code. This setting will not take effect unless at least one status check is enabled.
- Do not require status checks on creationAllow repositories and branches to be created if a check would otherwise prohibit it.
- Required status checks
  No checks have been added
  Learn more about status checks

Block force pushesPrevent users with push access from force pushing to refs.
Require code scanning resultsChoose which tools must provide code scanning results before the reference is updated. When configured, code scanning must be enabled and have results for both the commit and the reference being updated.
Require code quality resultsChoose which severity levels of code quality results should block pull request merges. When configured, a code quality analysis must be done on the pull request before the changes can be merged.
Restrict code coverageEnforce minimum line coverage thresholds on pull requests. This rule evaluates uploaded coverage data but does not wait for coverage uploads. To ensure coverage is evaluated before merging, make each status check associated with a coverage upload a required status check.
Automatically request Copilot code reviewRequest Copilot code review for new pull requests automatically if the author has access to Copilot code review and AI credits.
Footer
© 2026 GitHub, Inc.
Footer navigation

- Terms
- Privacy
- Security
- Status
- Community
- Docs
- Contact
- Manage cookies
- Do not share my personal information

cuales configs van para que al menos haya una aprobacion>?

Claude 17/10/2026 20:48

Para que nada entre a `main` sin al menos una aprobación, configurá así:

Datos generales

- **Ruleset Name:** `proteger-main`
- **Enforcement status:** `Active`
- **Bypass list:** dejala **vacía**. Así vos también, como dueño del repo, tenés que pasar por PR, y todos trabajan con la misma regla.

Target branches

- **Add target → Include default branch** (que es `main`).

Reglas que van tildadas

- ✅ **Restrict deletions**: nadie puede borrar `main`.
- ✅ **Block force pushes**: nadie puede reescribir el historial. Esto protege el antes/después.

- ✅ **Require a pull request before merging**. Al tildarlo aparecen sub-opciones:
  - **Required approvals:** 1
  - ✅ **Dismiss stale pull request approvals when new commits are pushed**: si alguien cambia código después de que se lo aprobaron, necesita aprobación de nuevo.
  - ✅ **Require conversation resolution before merging**: no se puede mergear con comentarios de review sin resolver. Esto les deja registrado que atendieron el feedback.
  - **Allowed merge methods:** dejá **Merge** habilitado. Evitá usar solo *Squash*, porque junta todos los commits de un PR en uno y pierden el commit por refactoring que les sirve de evidencia.

Reglas que van destildadas

- **Restrict creations** y **Restrict updates**: bloquearían a todos.
- **Require linear history**: choca con los merge commits.
- **Require signed commits**, **deployments**, **status checks**, **code scanning**, **code quality** y **code coverage**: no tienen esa infraestructura configurada.
- **Automatically request Copilot code review**: el review tiene que ser de ustedes. Si igual lo usan, va declarado en el registro de IA.

Dos aclaraciones

- GitHub no deja aprobar tu propio PR, así que con 1 aprobación cada PR lo tiene que aprobar otro integrante. Es justo lo que buscan.
- En cuentas gratuitas, los rulesets **solo se aplican si el repo es público**. Si lo tienen privado, la regla se guarda pero no bloquea nada. Como el docente igual tiene que poder entrar, lo más simple es que sea público.

Usuario (Cristian Morales) 17/10/2026 20:58

Si quieren, les preparo el `.gitattributes`, el `requirements-dev.txt` y la plantilla de PR para que los suban en el primer commit, junto con el README corregido.

hagamos esto

Claude 17/10/2026 20:59

Ejecutó un comando: Ver archivos del proyecto

Ejecutó un comando: Crear carpeta para los archivos de configuración

Creó el archivo `.gitattributes`

Creó el archivo `requirements-dev.txt`

Creó el archivo `pull_request_template.md`

Creó el archivo `README.md`

Ejecutó un comando: Empaquetar los archivos de configuración del repo

Envió el archivo `setup-repo.zip`

Están todos en **setup-repo.zip**:

- **`README.md`** reemplaza al actual. Tiene la instalación para PowerShell, CMD y Linux/Mac, la tabla de integrantes (completen los nombres y usuarios que faltan), cómo correr los tests y la forma de trabajo con ramas, PRs y tags.
- **`.gitattributes`** normaliza los finales de línea.
- **`requirements-dev.txt`** agrega pytest y httpx.
- **`.github/pull_request_template.md`** completa sola la descripción de cada PR nuevo.
- **`docs/PLAN_DE_TRABAJO.md`** es el plan de antes, ya ubicado en su carpeta.

Cómo subirlo

Como `main` ya está protegida, esto entra con un PR. Les sirve además para probar que la regla funciona.

En PowerShell, desde la carpeta del repo:

```
git checkout main
git pull
git checkout -b chore/setup-repo
```

Descomprimí el zip en la raíz del repo, aceptando reemplazar el README, y después:

```
git add .
git add --renormalize .
git status
```

Revisá el `git status` antes de seguir. Tienen que aparecer solo los archivos nuevos y el README. Si aparece `main.py` como modificado, es por los finales de línea y no por el código. En ese caso no lo incluyas en este commit: sacalo con `git restore --staged main.py` y lo vemos aparte. La idea es que `main.py` siga igual que en `estado-inicial`.

Si está todo bien:

```
git commit -m "chore: configuración del repo, README multiplataforma y plantilla de PR"
git push -u origin chore/setup-repo
```

Después abrí el PR en GitHub y que lo apruebe otro del grupo.

Antes de todo esto, confirmá que el tag `estado-inicial` ya esté subido. Lo ves en la pestaña **Tags** del repo.

Usuario (Cristian Morales) 17/10/2026 21:03

```
-r requirements.txt
```

quie ahce el -r

Claude 17/10/2026 21:03

El `-r` significa "incluí también lo que dice este otro archivo".

Entonces, cuando alguien corre:

```
pip install -r requirements-dev.txt
```

pip instala primero todo lo de `requirements.txt` (fastapi y uvicorn) y después lo propio del archivo dev (pytest y httpx).

Así no repiten las dependencias en los dos archivos. Si mañana agregan una librería al sistema, la ponen solo en `requirements.txt` y el entorno de desarrollo la incluye automáticamente.

Es el mismo `-r` que usan en la terminal con `pip install -r requirements.txt`, solo que escrito adentro del archivo.

Usuario (Cristian Morales) 17/10/2026 21:06

se puede armar un script para correr de forma mas simple el proyecto?

Claude 17/10/2026 21:07

Sí. Armo uno para Windows y otro para Linux/Mac. La primera vez crean el entorno e instalan las dependencias, y las siguientes veces solo levantan el sistema. Si alguien cambia los `requirements`, reinstalan solos.

Creó el archivo `run.ps1`

Creó el archivo `run.bat`

Creó el archivo `run.sh`

Lo pruebo en un proyecto completo de punta a punta.

Ejecutó un comando: Probar el script de Linux con todas sus opciones

Editó el archivo `run.ps1`

Editó el archivo `.gitattributes`

Editó el archivo `README.md`

Ejecutó un comando: Ajustar finales de línea de Windows y reempaquetar

Envió el archivo `setup-repo.zip`

Actualicé **setup-repo.zip** con los scripts. Reemplaza al anterior y va en la misma rama `chore/setup-repo`.

Cómo se usa

Para	Windows	Linux / Mac
Levantar el sistema	`.\run.bat` (o doble clic)	`./run.sh`
Correr los tests	`.\run.bat test`	`./run.sh test`
Volver a los datos iniciales	`.\run.bat reset`	`./run.sh reset`

La primera vez crea el `.venv` e instala todo. Las siguientes solo levanta el sistema.

En Windows usen **`run.bat`**: llama al `run.ps1` saltando el bloqueo de scripts de PowerShell, así no hace falta el `Set-ExecutionPolicy`.

Qué probé y qué no

- **`run.sh`:** lo probé completo, incluyendo la primera instalación, el arranque sin reinstalar, el sistema respondiendo, el reset y una opción inválida.
- **`run.ps1` / `run.bat`:** no tengo PowerShell acá, así que no los pude ejecutar. Los revisé a mano y hacen lo mismo que el de Linux. **Probalo vos primero** en tu máquina antes de pasárselo al grupo. Si algo falla, pasame el error.

Al subirlo

El README ahora tiene una sección **Uso rápido** con esta tabla.

Si alguien del grupo usa Linux o Mac, agregá además el permiso de ejecución del `.sh`, que desde Windows no se guarda solo:

```
git update-index --chmod=+x run.sh
```

Usuario (Cristian Morales) 17/10/2026 21:11

bnien quedo todo configurado, armamos una rama default dev para trabajar ahi y cuando tengamos todo resuelto pasarlo a main con el tag de estadorefactorizado

Claude 17/10/2026 21:12

Buena decisión: `main` queda limpio con la versión base hasta el final, y `dev` concentra todo el trabajo. Hay que ajustar algunas cosas para que funcione bien.

1. Revisar la protección (importante)

El ruleset apunta a **"default branch"**. Como ahora la rama por defecto es `dev`, **`main` dejó de estar protegida** y quedó protegida `dev` en su lugar.

Para proteger las dos, en **Settings → Rules → Rulesets → proteger-main → Target branches**:

- Quitá **Default branch**.
- **Add target → Include by pattern** → `main`.
- **Add target → Include by pattern** → `dev`.

Así ninguna de las dos acepta pushes directos y todo entra por PR con aprobación.

2. Cómo trabaja cada uno ahora

Todas las ramas salen de `dev` y vuelven a `dev`:

```
git checkout dev
git pull
git checkout -b refactor/registrar
```

Al abrir el PR en GitHub, la base va a ser `dev` automáticamente porque es la rama por defecto. Igual conviene mirarla antes de crearlo.

Quien ya tenía el repo clonado de antes tiene que traer la rama nueva:

```
git fetch
git checkout dev
```

3. Tags durante el trabajo

Los tags de etapa (`etapa-diagnostico`, `etapa-refactoring`, etc.) se ponen sobre commits de `dev`, en el momento en que cierra cada fase:

```
git checkout dev
git pull
git tag etapa-diagnostico
git push origin etapa-diagnostico
```

4. Cierre del trabajo

1. Abrir un PR de **`dev` → `main`** y que lo apruebe alguien del grupo.
2. Mergearlo con **Merge** (no squash), para que `main` conserve todo el historial de commits.
3. Poner el tag final sobre `main`:

```
git checkout main
git pull
git tag estadorefactorizado
git push origin estadorefactorizado
```

4. **Volver a poner `main` como rama por defecto** (Settings → General → Default branch). Cuando el docente abra el repo, va a ver la rama por defecto, y tiene que encontrar la versión final.

5. Textos a corregir

El README y el `PLAN_DE_TRABAJO.md` hablan de actualizar las ramas desde `main` y de que `main` está protegida. Hay que cambiarlo a `dev`. Si quieren, lo ajusto y les paso los dos archivos.

Usuario (Cristian Morales) 17/10/2026 21:13

no ajuntamos el plan de trabajo, pero nos sirve a nosotros, ya hice los ajustes en el readme sobre uqe main y dev estan protegidas

Claude 17/10/2026 21:13

Perfecto. Un detalle: el README que les pasé tiene esta línea al final de **Forma de trabajo**:

> El plan de trabajo, el checklist de code review y las evidencias están en la carpeta `docs/`.

Si el plan no va al repo, sacá "el plan de trabajo" de esa frase. Así nadie, ni el docente, busca un archivo que no existe.

Para el uso interno de ustedes, donde el plan dice `main` (actualizar la rama, base de los PR), léanlo como `dev`.

Con eso termina la Fase 0. Lo siguiente es la Fase 1: que cada uno lea el `main.py` completo, pruebe sus endpoints desde `/docs` y empiece a anotar smells de su parte. Cuando tengan los primeros, si quieren, los revisamos juntos.

Usuario (Cristian Morales) 17/10/2026 21:17

pasame mejor el readme corregido con estos ultimos cambios

Claude 17/10/2026 21:17

Editó el archivo `README.md`

Editó el archivo `README.md`

Envió el archivo `README.md`

El **README.md** corregido ya está listo. Los cambios están todos en la sección **Forma de trabajo**:

- Qué es `main` y qué es `dev`, y que las dos están protegidas.
- El flujo de trabajo: cada rama sale de `dev` y vuelve a `dev`, y al final `dev` pasa a `main`.
- Dónde va cada tag: los de etapa sobre `dev` y `estadorefactorizado` sobre `main`.
- Saqué la mención al plan de trabajo. Ahora dice que en `docs/` están el checklist de review, las decisiones técnicas y las evidencias.

El resto quedó igual. Como ya habían tocado el README, reemplazalo entero por este y revisá en el diff del PR que no se haya perdido nada que hayan agregado ustedes, por ejemplo los usuarios de GitHub en la tabla de integrantes.

Usuario (Cristian Morales) 17/10/2026 21:26

bien, como sugeris armar la estuctura de carpetas y archivos para evitar conflictos?

Claude 17/10/2026 21:26

Antes de la propuesta, una aclaración: la estructura final es una **decisión de diseño que tiene que salir de su diagnóstico**. Cada carpeta debería poder justificarse con un smell que resuelve y un principio que aplica. Les dejo una propuesta como punto de partida; la validan contra lo que encuentren en la Fase 1 y la dejan escrita en `docs/` como decisión técnica.

Idea central para evitar conflictos

Hoy los cuatro trabajan sobre **un solo archivo**, así que cualquier cambio en paralelo choca. La solución es que cada uno trabaje sobre **archivos distintos**. Para eso, la estructura se separa en dos ejes:

- **Por capa:** rutas, lógica de negocio, acceso a datos.
- **Por funcionalidad:** que coincida con el reparto de A, B, C y D.

Propuesta de estructura

```
app/
├── main.py                 # Crea la app y registra los routers          (D)
├── config.py               # Configuración: ruta de la base, etc.        (D)
├── database.py             # Conexión e inicialización de tablas         (D)
├── seed.py                 # Datos de prueba                             (D)
│
├── domain/                 # Reglas y conceptos del negocio
│   ├── estados.py          # Estados y transiciones                      (B)
│   └── clasificacion.py    # Tipos, prioridad y área                     (A)
│
├── schemas/                # Modelos de entrada y salida
│   ├── personas.py         # Estudiante y Responsable                    (D)
│   └── solicitud.py        # Solicitud                                   (A)
│
├── repositories/           # Acceso a datos (todo el SQL vive acá)
│   ├── estudiante_repo.py  #                                             (D)
│   ├── responsable_repo.py #                                             (D)
│   ├── solicitud_repo.py   # Compartido: D crea la base, cada uno agrega
│   └── historial_repo.py   #                                             (B)
│
├── services/               # Lógica de negocio
│   ├── registro.py         # Registrar solicitud                         (A)
│   ├── gestion.py          # Cambiar estado y asignar responsable        (B)
│   ├── consultas.py        # Búsqueda y detalle                          (C)
│   ├── reportes.py         # Reporte                                     (C)
│   └── notificaciones.py   # Avisos al estudiante                       (A crea, B usa)
│
└── routers/                # Endpoints (solo reciben, delegan y responden)
    ├── estudiantes.py      #                                             (D)
    ├── responsables.py     #                                             (D)
    ├── solicitudes_registro.py   # POST /solicitudes                     (A)
    ├── solicitudes_gestion.py    # PUT estado y responsable              (B)
    ├── solicitudes_consultas.py  # GET /solicitudes y /solicitudes/{id}  (C)
    └── reportes.py         # GET /reporte                                (C)

tests/
├── conftest.py             # Base de datos limpia para cada test         (C)
├── test_registro.py        #                                             (A)
├── test_gestion.py         #                                             (B)
├── test_consultas.py       #                                             (C)
├── test_reportes.py        #                                             (C)
└── test_personas.py        #                                             (D)
```

FastAPI permite registrar varios routers con el mismo prefijo `/solicitudes`. Por eso las rutas de solicitudes se pueden repartir en tres archivos, uno por integrante, sin que cambie ninguna URL.

Orden para migrar sin romper nada

Paso 1 — Esqueleto (D, un solo PR, antes que el resto):

- Crear todas las carpetas con archivos vacíos o mínimos.

- `config.py`, `database.py` y `seed.py` con lo que hoy está al principio de `main.py`.
- `app/main.py` registrando **todos** los routers desde el inicio, aunque estén vacíos. Así nadie tiene que tocar ese archivo después, que sería el punto de conflicto más probable.
- `solicitud_repo.py` con lo mínimo que usan todos (buscar por id, convertir una fila).
- Actualizar `run.ps1`, `run.sh` y el README: el comando pasa a ser `uvicorn app.main:app`.
- Los tests tienen que seguir en verde.

Paso 2 — Cada uno migra su parte (en paralelo):

- Saca su endpoint del `main.py` viejo y lo lleva a su router, su servicio y su repositorio.
- **Un endpoint por vez**, corriendo todos los tests después de cada uno.
- En `solicitud_repo.py`, cada uno agrega sus funciones **al final del archivo**. Si hay conflicto, es trivial de resolver.

Paso 3 — Cierre:

- Cuando el `main.py` viejo quede vacío, se borra en un PR propio.

Dos advertencias

- **No sobrediseñen.** La consigna evalúa la justificación, no la cantidad de carpetas. Si una capa no resuelve ningún smell de su tabla, sáquenla. Por ejemplo, si `domain/` les queda con una sola constante, puede ir dentro de `services/`.
- **Los tests no se mueven de lugar ni cambian de lógica** durante la migración. Prueban los endpoints desde afuera, así que tienen que pasar igual antes y después. Esa es justamente la evidencia de que el refactor fue seguro.

Usuario (Cristian Morales) 17/10/2026 21:34

de este plan de trabajo cambia los integrantes a:

A - Gaston
B - Javier
C - Bruno
D - Cristian

[Archivo adjunto: PLAN_DE_TRABAJO.md]

Claude 17/10/2026 21:35

Ejecutó un comando: Reemplazar las letras por los nombres del grupo

Frené ahí: el reemplazo no se aplicó y el archivo sigue sin cambios.

Decime cómo preferís seguir.

Usuario (Cristian Morales) 17/10/2026 21:35

Rey, del plan de trabajo que nos armaste, cambia los significados de A B C y D con los nombres propuestos

Claude 17/10/2026 21:36

Creó el archivo `PLAN_DE_TRABAJO.md`

Envió el archivo `PLAN_DE_TRABAJO.md`

En **PLAN_DE_TRABAJO.md** ya están los nombres en lugar de las letras:

- **Gaston**: registrar solicitud. Coordina la tabla de smells y garantiza el Introduce Parameter Object.
- **Javier**: cambio de estado y asignación. Coordina el code review y `/docs`, y garantiza el Move Method.
- **Bruno**: búsqueda, detalle y reporte. Coordina los tests y garantiza el Extract Method.

- **Cristian**: base de datos, estudiantes y responsables. Coordina el informe, los tags y el video, y hace los cambios transversales.

La rotación de review queda así: Gaston revisa a Javier, Javier a Bruno, Bruno a Cristian y Cristian a Gaston.

Además del reemplazo, saqué la nota que pedía cambiar las letras y el párrafo sobre qué hacer si eran 3 integrantes, porque ya no aplican. En la tabla de registro de IA dejé una fila para cada uno.

Las dos menciones a "actualizar la rama desde `main`" quedaron como estaban. Si quieren que diga `dev`, lo cambio.

Usuario (Cristian Morales) 17/10/2026 21:37

la propuesta de la estructura ya podriamos configurarla entonces, previo a esto, hay que armar test?

Claude 17/10/2026 21:38

Sí, primero van los tests. Y antes de los tests, el diagnóstico. El orden es este:

1. **Diagnóstico (Fase 1).** Cada uno detecta los smells de su parte y arman la tabla en conjunto.
2. **Tests (Fase 2).** Se escriben y se corren sobre el código actual, sin tocarlo, y tienen que dar todos en verde.
3. **Estructura y refactoring (Fase 3).** Recién acá se arma la estructura de carpetas.

Por qué en ese orden

- **Armar la estructura ya es refactorizar.** Mover código de `main.py` a otros archivos es un Move Method o una reubicación de responsabilidad. Si lo hacen antes de tener tests, no pueden demostrar que no rompieron nada, y eso es justamente lo que evalúa el criterio de "refactoring seguro" (20%).
- **La estructura tiene que salir del diagnóstico.** En el informe van a tener que justificar cada carpeta con un smell de la tabla. Si la arman antes, la justificación queda armada al revés: primero la solución y después el problema.
- **Los tests son la evidencia del "antes".** Si los escriben después de cambiar el código, ya no prueban el sistema original.

Qué sí pueden ir preparando ahora

Sin tocar `main.py`:

- **Bruno** puede armar la base de los tests: la carpeta `tests/` y un `conftest.py` que levante el sistema con datos limpios en cada test. Esto no modifica el sistema, solo lo prueba desde afuera. Va a tener que resolver cómo lograr una base limpia sin cambiar `main.py`; si se traba, lo vemos.
- **Gaston** puede compartir la plantilla de la tabla de smells.
- **Javier** puede proponer el borrador del checklist de code review.
- **Todos** pueden arrancar con la lectura y el diagnóstico de su parte.

Si quieren tener a mano la propuesta de estructura, pueden guardarla como borrador en `docs/`. Después la ajustan con lo que salga del diagnóstico y la dejan como decisión técnica definitiva antes de la Fase 3.